UNIVERSIDADE DE SÃO PAULO
ESCOLA DE ENGENHARIA DE SÃO CARLOS

Programa Institucional de Bolsas de Iniciação Científica (PIBIC)

Relatório Final

Implementação e aplicações de redes neurais hierárquicas de aprendizado profundo para cálculo de interpolações por elementos finitos

Autor: Pedro Azevedo Coelho Carriello Corrêa
Orientador: Dr. Pablo Giovanni Silva Carvalho

São Carlos, SP
Agosto, 2024

Agradecimentos

Este pesquisador dirige seus sinceros agradecimentos ao Dr. Pablo Giovanni Silva Carvalho, que orientou-o com seu conhecimento em elementos finitos e ajuda na pesquisa bibliográfica; ao Prof. Dr. Fabrício Simeoni de Sousa, que cedeu o acesso ao seu laboratório no Instituto de Ciências Matemáticas e de Computação para a realização da pesquisa; ao Conselho Nacional de Desenvolvimento Científico e Tecnológico, pela concessão da bolsa PIBIC; e aos Dr. Lei Zhang e Ph. D. Wing K. Liu, autores de vários artigos relevantes na área de *Physics-Informed Neural Networks* e que gentilmente reponderam dúvidas sobre vários pontos do seu *framework* de redes neurais hierárquicas de aprendizado profundo, o que foi base para a implementação do algoritmo aqui utilizado.

Sumário

1 Introdução e Motivação . . . 3
1.1 Mudança de direcionamento da pesquisa . . . 3
2 Objetivos . . . 4
3 Metodologia . . . 4
3.1 Redes Neurais *Multi-Layer Perceptron* . . . 5
3.2 Método de Elementos Finitos . . . 5
3.3 Redes Neurais de Aprendizado Profundo Hierárquicas (HiDeNN) . . . 7
3.4 Primeira versão do treinamento do modelo . . . 8
3.5 Versão final do treinamento do modelo . . . 9
3.6 Otimizador *Adam* . . . 10
4 Atividades e Resultados . . . 11
4.1 Código para implementação de Método de Elementos Finitos com diferentes funções de forma . . . 11
4.2 Versão 1 da rede neural de aprendizado profundo hierárquica . . . 14
4.3 Exemplo de aplicação da HiDeNN versão final . . . 15
4.3.1 Implementação . . . 16
4.3.2 Resultados . . . 17
5 Conclusão . . . 22

REFERÊNCIAS . . . 23

1 Introdução e Motivação

Desde o início da criação dos computadores, os que haviam de mais modernos já eram utilizados para a resolução de equações complexas. Na verdade, na época, essa era a função primária desses dispositivos. O cálculo para melhorar a acurácia dos projéteis, durante a Segunda Guerra Mundial, passou a tentar considerar, entre outros fatores, viagens de dezenas de quilômetros, resistência do vento, mudanças da força gravitacional dependendo da altura alcançada, o que tornaram as soluções analíticas das equações diferenciais cada vez mais difíceis. Nesse contexto, utilizavam computadores como o Analisador Diferencial, mecânico analógico, ou a Máquina de Mallock, elétrico analógico, para tentar encontrar essas soluções (CROARKEN, 2002).

Atualmente, com uma grande expansão do uso de Inteligência Artificial em diversos setores, os métodos de aprendizado de máquina são a mais nova tecnologia a se posicionar na vanguarda das ferramentas para a análise numérica. Na análise em áreas reconhecidas pela complexidade de suas equações, como a de mecânica dos fluidos, um dos métodos que se destaca é o das Redes Neurais Artificiais, em especial as de *Multi-Layer Perceptron* (SHARMA et al., 2023).

Nessa pesquisa, são utilizadas redes neurais do tipo *multi-layer perceptron* para aperfeiçoar o método de elementos finitos aplicados em um contexto ilustrativo de mecânica dos sólidos e analisar, até certo escopo, a acurácia das soluções encontradas e a influência de parâmetros relevantes para a escolha do uso do método.

1.1 Mudança de direcionamento da pesquisa

No início da pesquisa bibliográfica, o foco era o estudo da implementação de redes neurais para o aprimoramento de solução de sistemas por meio do Método de Diferenças Finitas. Porém, após um estudo inicial desse método (baseado em Langtangen e Linge (2018)) e do Método de Elementos Finitos (a partir de Becker e etc. (1981a)), da formação de um grupo de estudo com professores e alunos de pós-graduação focados na biblioteca `FEniCSx` (que se baseia em elementos finitos), uma decisão foi feita para uma mudança na metodologia do projeto. Por contar com o apoio dos integrantes do grupo de estudos de elementos finitos, e após a constatação de que a mudança de foco no estágio inicial do projeto seria possível, a alteração foi considerada uma decisão sensata pelo aluno e pelo orientador da pesquisa.

Tanto o de diferenças finitas, quanto o de elementos finitos, são métodos de solução numérica extensivamente aplicados e consolidados historicamente no contexto de mecânica (THOMÉE, 1984) e, no contexto de aprendizado de máquina, as redes neurais artificiais já se provaram como uma ferramenta capaz de aperfeiçoar ambos os métodos, como se pode verificar em Pantidis e Mobasher (2023), Meethal et al. (2023), Le-Duc, Nguyen-Xuan e

Lee (2023) para elementos finitos e Tu e Nguyen (2022), Shi et al. (2020) para diferenças finitas. Assim, a mudança da metodologia da pesquisa ainda conserva o uso de métodos já consolidados para soluções numéricas.

2 Objetivos

Inspirado nos artigos Saha et al. (2021) e Zhang et al. (2021b), o projeto visa a formulação e o teste de um método que utiliza uma rede neural hierárquica para realimentar a localização dos nós de elementos finitos. Essa realimentação viria do treinamento da rede e, a partir dela, é esperado que os elementos finitos ficam mais refinados nas regiões de maior sensibilidade da solução, ou seja, os nós iriam se concentrar nas regiões que a solução mais flutua, de modo a diminuir seu erro, enquanto regiões mais lineares da solução não concentrariam tantos elementos.

3 Metodologia

Os principais modelos de redes neurais e métodos de aprendizado de máquina foram introduzidos ao aluno por meio do curso das disciplinas SCC0230 - Inteligência Artificial e SCC0270 - Redes Neurais e Aprendizado Profundo, ministradas, respectivamente, pela Profa. Solange Oliveira Rezende e Prof. Moacir Antonelli Ponti. Posteriormente, foi aprofundado o estudo de PINNs (*Physics-Informed Neural Network*) a partir de artigos como o Sharma et al. (2023) e vídeo-aulas na internet. Já o método de elementos finitos foi estudado principalmente a partir da leitura de Becker e etc. (1981a).

Já a implementação do algoritmo foi inicialmente em `Python`, principalmente através do uso da biblioteca `PyTorch` e de bibliotecas auxiliares, como a `Matplotlib` e `NumPy`. Além destas, houve o estudo da biblioteca `FEniCSx`, incluindo estudos em conjunto com outros pesquisadores e a realização de lições de *jupyter notebooks* disponibilizados por Baratta (2023), mas, em dado ponto do projeto, foi decidido focar no uso da `PyTorch` e descartar a `FEniCSx`. Porém, após certas dificuldades no uso do `PyTorch`, principalmente no seu pacote `autograd`, a implementação do algoritmo foi migrada para o MATLAB, sendo que, neste, a implementação do treino da rede foi feita manualmente, sem o uso de demais bibliotecas.

O estudo da solução do algoritmo seu deu aplicando-o em uma equação exemplo de mecânica dos sólidos e comparando-a com a solução exata. Foi alterado alguns parâmetros, e, para cada combinação, computado o erro. O computador utilizado foi um notebook com processador Intel® Core™ i5-1235U @1300MHz e GPU Intel® Iris® Xe Graphics.

Nas próximas subseções, serão explicados conceitos relevantes para a metodologia aplicada.

3.1 Redes Neurais *Multi-Layer Perceptron*

Redes neurais de aprendizado profundo são redes que tentam superar limitações de modelos lineares *perceptron* incorporando mais camadas. A forma mais natural de fazer isso é alocando camadas conectadas em seguida, de forma que cada camada anterior alimenta a próxima, até gerar a saída do modelo. Essa arquitetura é chamada *multilayer perceptron* (MLP) (ZHANG et al., 2021a).

No geral, redes desse tipo são modelos de aprendizado supervisionado, ou seja, são treinadas a partir de um banco de dados contendo entradas e suas respectivas saídas esperadas. Porém, nesse projeto foi feita uma modificação em relação ao modelo *perceptron* tradicional. Nesse algoritmo, a função de perda (*Loss Function*) é calculada a partir de equações que devem ser minimizadas dadas pela física do problema, no geral, equações de energia potencial. Tal abordagem é mais explicada na Seção 3.5 e é comumente nomeada como Redes Neurais Informadas Pela Física (*Physics-Informed Neural Networks*).

3.2 Método de Elementos Finitos

Para formular o método de elementos finitos, considera-se a equação genérica governante no domínio $x_0 < x < x_L$:

$$-k\frac{\mathrm{d}u(x)^2}{\mathrm{d}^2x} + c\frac{\mathrm{d}u(x)}{\mathrm{d}x} + bu(x) = f(x) \tag{1}$$

Onde k, c e b são coeficientes fixos e $f(x)$ uma função de x dados pela equação que se queira resolver. Considerando condição de borda de Dirichlet onde $u(x_0) = u_0$ e $u(x_L) = u_L$, e $v(x)$ uma função de teste definida em todo o intervalo, pode-se analisar a Equação 1 pelo ponto de vista do método de elementos finitos e reescrevê-la na forma variacional:

$$\int_{x_0}^{x_L}(ku'v' + cu'v + buv)\mathrm{d}x = \int_{x_0}^{x_L}(fv)\mathrm{d}x \tag{2}$$

Nesse contexto, v e u para satisfazerem corretamente o enunciado devem pertencer ao subespaço H^1, definido pelas funções cujas integrais da Equação 3 convergem.

$$\int_{x_0}^{x_L}\left[(v')^2 + v^2\right]\mathrm{d}x < +\infty \tag{3}$$

Dividi-se, arbitrariamente, o domínio do problema em N elementos finitos, cada elemento i com h_i de comprimento. Após isso, são construídas uma função de forma ϕ_i para cada elemento e que geram uma base para o subespaço H^h de H^1: $\{\phi_1, \phi_2, \ldots, \phi_N\}$.

Assim, procura-se uma função $u_h \in H_h$ que possa ser escrita como uma interpolação das funções de forma:

$$u_h(x) = \sum_{j=1}^{N}\alpha_j\phi_j(x) \tag{4}$$

Escrevendo v_h da mesma forma e substituindo na Equação 2, temos:

$$\int_{x_0}^{x_L}(ku'_hv'_h + cu'_hv_h + bu_hv_h)\mathrm{d}x = \int_{x_0}^{x_L}(fv_h)\mathrm{d}x$$

Equivalentemente:

$$\sum_{j=1}^{N} K_{ij}\alpha_j = F_i\,, \quad i = 1, 2, \ldots N \tag{5}$$

Sendo $\mathbf{K} = [K_{ij}]$ comumente chamada de matriz de rigidez, e $\mathbf{F} = \{F_i\}$, de vetor de carga.

$$K_{ij} \coloneqq \int_{x_0}^{x_L}(k\phi'_i\phi'_j + c\phi'_i\phi_j + b\phi_i\phi_j)\mathrm{d}x \tag{6}$$

$$F_i \coloneqq \int_{x_0}^{x_L}(f\phi_i)\mathrm{d}x \tag{7}$$

Com $1 \leq i, j \leq N$.

Considerando que essas matrizes são conhecidas, é possível resolver a Equação 5 para os coeficientes α_i, e utilizar a Equação 4 para obter a aproximação de Galerkin para a solução de $u(x)$. E essa é a apresentação básica do método dos elementos finitos. Todo esse processo está descrito mais aprofundadamente em Becker e etc. (1981b).

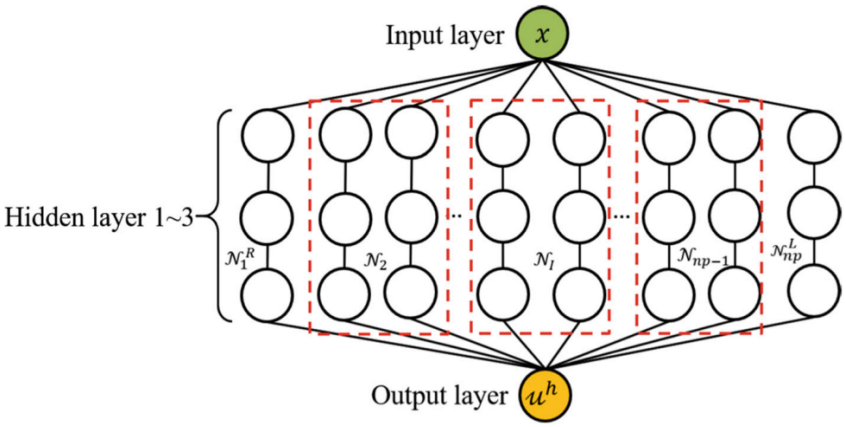
Algo interessante é que há poucas restrições na escolha da função utilizada como ϕ_i, o que torna esse método bastante flexível. As únicas restrições são referentes a pertencer ao espaço H^1 (Equação 3) e cumprir a Equação 8. Apesar dessa grande liberdade de escolha para essa função, as mais utilizadas continuam sendo a linear e a quadrática por, dentre outros motivos, a sua facilidade de implementação (NETZ; DÜSTER; HARTMANN, 2013). Nesse estudo, é utilizada a função de forma linear, a mais simples, já que aqui o foco não foi o aprofundamento tão grande do método de elementos finitos, e sim, observar como este pode ser aperfeiçoado por redes neurais.

$$\phi_i(x_j) = \begin{cases} 1, & \text{se } i = j \\ 0, & \text{se } i \neq j \end{cases} \tag{8}$$

A função de forma linear, ou chapéu, é definida de acordo com a Equação 9, considerando h_i o tamanho de cada elemento finito, que pode ser variável. A partir dessa equação, é possível fazer uma análise da função nos nós e chegar na conclusão que esta cumpre a Equação 8.

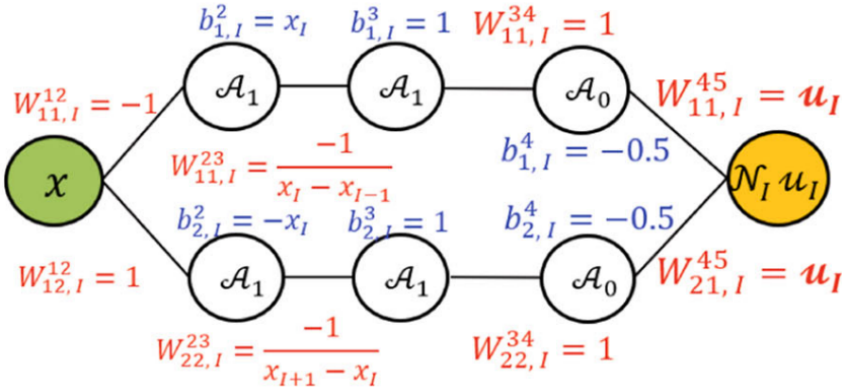
$$\phi_i(x) = \begin{cases} \dfrac{x - x_i}{h_i}, & \text{para } x_{i-1} \leq x \leq x_i \\ \dfrac{x_{i+1} - x}{h_{i+1}}, & \text{para } x_i \leq x \leq x_{i+1} \\ 0, & \text{para } x \leq x_{i-1} \text{e } x \geq x_{i+1} \end{cases} \tag{9}$$

Figura 1 – Estrutura básica da rede neural de aprendizado profundo hierárquica do projeto



Fonte: Zhang et al. (2021b)

Figura 2 – Estrutura da rede neural $\mathcal{N}_I$, de um nó no interior do domínio



Fonte: Zhang et al. (2021b)

Pelas Equações 8 e 4, pode-se concluir que $u_I := u(x_i) = \alpha_i$ e, assim, tem-se a Equação 10.

$$u_h = \sum_{i=1}^{N} u_I \phi_i(x) \tag{10}$$

3.3 Redes Neurais de Aprendizado Profundo Hierárquicas (HiDeNN)

Inspirado em Saha et al. (2021) e Zhang et al. (2021b), são implementadas redes neurais para cada nó e estas são colocadas em paralelo, assim como ilustrado na Figura 1. As redes de cada nó são as $\mathcal{N}_1^R, \mathcal{N}_2, \ldots, \mathcal{N}_I, \ldots, \mathcal{N}_{np-1}, \mathcal{N}_{np}^L$, sendo np o número de nós que discretiza o domínio.

Os nós do interior possuem as camadas ilustradas na Figura 2. Vale ressaltar que os parâmetros de peso $W_{11,I}^{12}$, $W_{12,I}^{12}$, $W_{11,I}^{34}$ e $W_{22,I}^{34}$, e os parâmetros de viés $b_{1,I}^3$, $b_{2,I}^3$, $b_{1,I}^4$ e $b_{2,I}^4$ são fixos. Já os outros pesos e viéses variam de acordo com as coordenadas dos nós e o valor da solução calculada para o nó (u_I).

Na Figura 2, $\mathcal{A}_0$ e $\mathcal{A}_1$ representam as funções de ativação de cada camada. A primeira, $\mathcal{A}_0 = x$, já $\mathcal{A}_1$ é uma função comumente chamada no campo de redes neurais de `ReLU`, definida no Listing 1.

Listing 1 – Definição da função `ReLU`

```
1    def ReLU(x):
2        return max(0,x)
```

As camadas ocultas da rede $\mathcal{N}_I$ podem ser interpretadas como geradoras da função de forma chapéu $\phi_i(x)$. Já a última camada realiza a multiplicação pelo termo u_I, o que, após a combinação de todas as redes hierárquicas, é análoga à interpolação das funções de forma, ou seja, a Equação 10.

3.4 Primeira versão do treinamento do modelo

A primeira tentativa de formulação da função de perda foi com base no resíduo da equação do problema considerando a substituição dos vetores u_h, u'_h e u''_h como aproximações de u, u' e u'', respectivamente. A rotina desenvolvida para geração da rede e treinamento segue os seguintes passos:

1. São inseridos como entradas uma equação no formato $-k \cdot u''(x) + c \cdot u'(x) + b \cdot u(x) = f(x)$, o domínio onde ela governa, e as condições de contorno de Dirichlet, i.e., os valores de $u(x)|_{x=x_0}$ e $u(x)|_{x=x_L}$.

2. O domínio é repartido em N nós e $N-1$ elementos finitos de tamanhos iguais de h de comprimento.

3. A partir da função de forma chapéu e a equação dada pelo problema, calculam-se as matrizes $\mathbf{K}$ e $\mathbf{F}$ baseadas no método de elementos finitos, exposto na Seção 3.2.

4. Para encontrar os coeficientes interpoladores das funções de forma, faz-se $\mathbf{K}\{u_I\} = \mathbf{F}$ e resolve para $\{u_I\}$.

5. Tendo a inicialização dos nós e coeficientes interpoladores feita, forma-se uma rede hierárquica, assim como exposta na Seção 3.3.

6. Para cálculo da perda propriamente dita:

   a) Utilizando a rede gerada, realiza-se um primeiro *forward* pela rede, ou seja, dado o domínio como entrada para a rede, retorna a sua saída, que é a primeria iteração de solução aproximada.

   b) A partir dessa solução, são aproximados os valores de u' e u'' e a função de perda é então dada por $l(x, \theta) = \text{MSE}(-ku'' + cu' + bu, f)$, sendo θ o conjunto

dos parâmetros atuais da rede, e MSE a função de erro quadrático médio (*Mean Squared Error*). Ou seja, a perda é calculada pelo resíduo da equação original utilizando a solução aproximada e com base no erro quadrático médio.

7. A partir da função de perda, utiliza-se um método do estado da arte (como o `Adam`) para calcular o *backpropagation* da rede e recalcular o valor dos nós dos elementos finitos.

8. Com esses novos nós, realiza novamente o passo em diante até um certo número de iterações ou até uma perda satisfatória.

Porém, essa metodologia não estava conseguindo atualizar as posições dos nós a fim de diminuir o erro. Nesse ponto, foi revisitada toda a teoria de elementos finitos e foi compreendido que esse comportamento indesejado se dava pois, matematicamente, a solução em elementos finitos já minimiza o resíduo dessa equação quando todos os termos envolvendo u são substituídos por u_h. O que poderia ser de fato utilizado como função de perda seria o resíduo entre a comparação da equação aproximada em relação à solução analítica. Porém, não faria muito sentido comparar ambas considerando que, no caso de conhecer-se a solução analítica, não teria porque utilizar uma aproximação, além de tornar o algoritmo utilizável apenas para casos onde já se sabe a solução, o que é bem restritivo.

3.5 Versão final do treinamento do modelo

Para chegar-se na formulação final do treinamento e da *loss function*, foi aprofundado o estudo de redes neurais informadas pela física (SHARMA et al., 2023). Assim, foi compreendido que a fórmula da perda deve ser definida de forma específica para cada problema em questão. Isso porque esta depende da equação que rege o sistema físico tratado.

No caso dos problemas aplicados na pesquisa, no campo da mecânica dos sólidos, a equação a ser minimizada, e que, portanto, pode ser tratada como a função de perda, é a de energia potencial. Isso porque, dado um sistema em equilíbrio, temos que a sua energia potencial assume um valor de mínimo (RAO, 2001).

Assim, os passos a serem efetuados para formular a função de perda dos problemas seguiu, basicamente, a rotina descrita no restante dessa Seção.

1. De forma semelhante à primeira versão, são inseridos os parâmetros de entrada: número de nós, domínio de governança e valor da equação no contorno do domínio. Além do número máximo de iterações, valores dos parâmetros do otimizador *Adam* (mais explicado na Seção 3.6) e parâmetros da quadratura de Gauss (as integrações necessárias no processo são executadas pelo método da quadratura).

2. É definida a função de forma utilizada.

3. A partir da equação de energia potencial, $J(u)$, dada pelo problema, deriva-a em relação a u e, como $J(u)$ espera-se ser minimizado, sua derivada pode ser igualada a 0. A partir da equação resultante e realizando as integrações por quadratura de Gauss, pode-se extrair uma equação semelhante à $\mathbf{Ku} = \mathbf{F}$, da onde retira-se os valores iniciais para os coeficientes interpoladores u_I.

4. Geração de uma iteração da rede neural.

5. Considerando $J(u)$ a função de perda da rede e que a rede tem seus parâmetros dependentes de $\mathbf{u}$, os deslocamentos de cada nó, e $\mathbf{x}$, a posição de cada nó, o cálculo das derivadas da função de perda em relação a ambos os vetores se faz necessário e, a partir desses gradientes, utiliza-se o otimizador *Adam* para ajustar os parâmetros da rede. Assim, são atualizados a posição dos nós e seus deslocamentos.

6. São repetidos os passos de recalcular a função de perda e otimização pelo *Adam* até atingir uma perda razoável.

7. Finalmente, o código recalcula os deslocamentos dos nós por elementos finitos, considerando as novas posições encontradas.

3.6 Otimizador *Adam*

O otimizador *Adam* (KINGMA; BA, 2014) é um algoritmo já bem consolidado e, em muitos contexto, considerado o estado da arte entre os otimizadores de redes neurais. Seu nome vem de estimador de momento adaptativo. Ele adiciona na equação termos inspirados no conceito de momentum, da física mecânica, de modo que altera o módulo de mudança dos parâmetros da rede ao, basicamente, iniciar com passos maiores e, ao aproximar-se de um ponto de mínimo, diminuir o tamanho destes.

No artigo que apresenta o algoritmo, tem-se $\nabla_\theta l_t(\theta_{t-1})$, o gradiente em relação aos parâmetros θ da função de perda l da iteração t calculada sobre os parâmetros da iteração $t-1$. Nesta pesquisa, otimizador é aplicado tanto para a atualização das posições dos nós, quanto dos seus deslocamentos. No caso das posições, tem-se o gradiente como $\nabla_{\mathbf{X}} J_t(\mathbf{X}_{t-1})$ e, no dos deslocamentos, $\nabla_{\mathbf{u}} J_t(\mathbf{u}_{t-1})$, que são, basicamente, $\frac{\partial J}{\partial \mathbf{X}}$ e $\frac{\partial J}{\partial \mathbf{u}}$.

Assim como recomendado em Kingma e Ba (2014), foram definidos os parâmetros do otimizador, inicialmente, da seguinte forma:

$$\begin{cases} \alpha = 0.001 \\ \rho_1 = 0.9 \\ \rho_2 = 0.999 \\ \epsilon = 10^{-8} \end{cases}$$

4 Atividades e Resultados

A primeira atividade realizada foi o estudo das teorias que envolviam o primeiro conceito do projeto, que posteriormente mudou, como descrito na Subseção 1.1. Assim, foi estudado o método das diferenças finitas por meio da leitura de Langtangen e Linge (2018) e, concomitantemente, o conceito de mínimos quadrados móveis e suas aplicações em diferenças finitas, como em Sousa et al. (2019). Durante o estudo dessas fontes, foram implementados códigos em *Python* para melhor entender como funcionam os métodos. Após a decisão de mudança no projeto, os novos estudos foram acompanhados com frequentes reuniões com o orientador da pesquisa e com o grupo de estudos referentes à biblioteca `FEniCSx`. Dentre esses novos códigos, vale mencionar:

- Realização de algumas lições de *jupyter notebooks* disponibilizados por Baratta (2023) para o exercício da biblioteca `FEniCSx`.
- Implementação de códigos para geração de aproximações por elementos finitos de equações de 1 dimensão (4.1).
- Primeira versão do código com aprimoramento pela rede hierárquica de aprendizado profundo em *Python* (4.2).

Até esse momento do projeto, um grande problema era a ainda não definição consistente do cálculo da perda da rede neural. Até esse ponto, era utilizada o *framework* para cálculo da perda descrito na Seção 3.4. Porém, pelos motivos descritos na mesma seção, foi descartada essa metodologia e retornou-se ao processo de formulação da *loss function*.

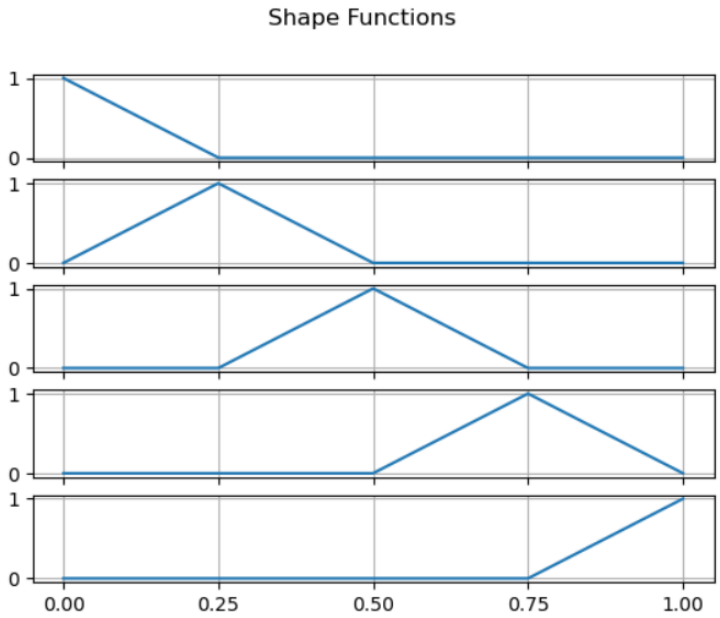
Com o esgotamento de algumas tentativas de reparação do código para o correto treinamento do modelo (dentre as tentativas, mudanças entre bibliotecas), o autor recorreu a pesquisadores já consolidados da área de redes neurais hierárquicas para esclarecer alguns pontos. Os pesquisadores Dr. Lei Zhang e Ph. D. Wing K. Liu, autores de artigos que estavam servindo de base para esta pesquisa (ZHANG et al., 2021b; ZHANG et al., 2021a), ajudaram na formulação do algoritmo de perda, explicado na Seção 3.5.

Após isso, o modelo foi aplicado em um exemplo e analisado o seu erro, processo exposto nas Seção 4.3.

4.1 Código para implementação de Método de Elementos Finitos com diferentes funções de forma

Foram elaborados códigos com um baixo uso de bibliotecas auxiliares, para forçar a implementação do método de elementos finitos quase do zero. Dentre os códigos, destaca-se

Figura 3 – Funções de forma geradas pelo código do Listing 2



Fonte: Elaborada pelo autor

um que recebe como entrada a equação do problema, o domínio, as condições de contorno, a quantidade de nós e o grau da função de forma desejada, e ele retorna a aproximação calculada considerando elementos de mesmo tamanho. Tal código pode ser chamado para resolver o problema de Poisson com função de forma chapéu, como exibido no Listing 2.

Listing 2 – Chamada da função para aplicada ao problema de Poisson

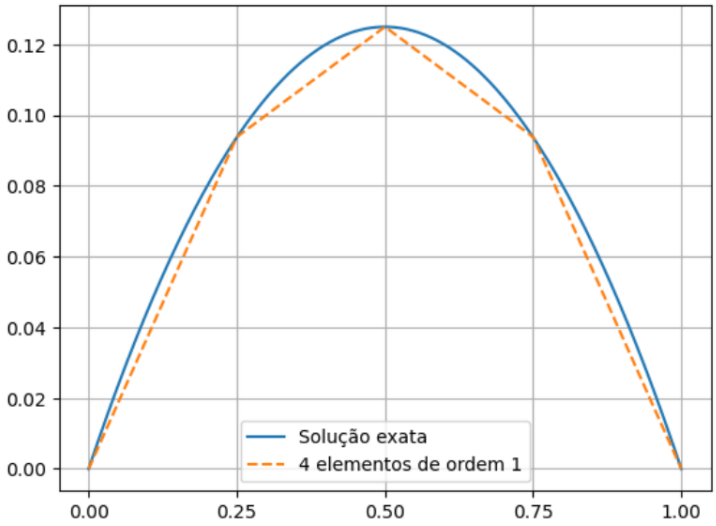
```
equation = [1, 0, 0, lambda x: 1] # [k, c, b, f(x)]
bound_cond = [0, 0] # condicoes de borda
dom = np.linspace (0,1,500) # dominio

u_aprox = FE1D_constructor (dom=dom,
    equation=equation,
    n=5,
    k=1,
    bound_cond=bound_cond)
```

Com essa entrada, o algoritmo é capaz de retornar as matrizes de rigidez e de carga do modelo, além de imprimir o gráfico das funções de forma e da comparação com a solução exata (Figura 3 e 4).

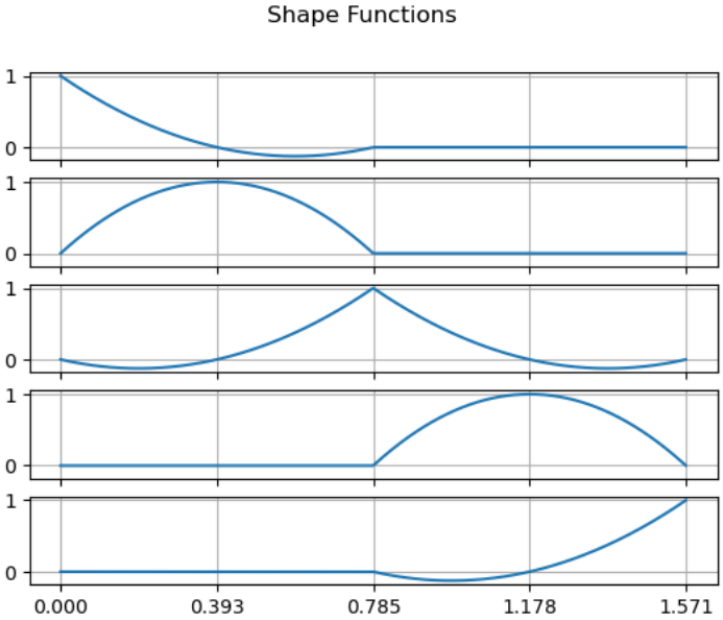
Com esse mesmo código foram geradas soluções aproximadas para outros problemas utilizando outras funções de forma, como exemplificado nas Figuras 5 e 6.

Figura 4 – Comparação da solução gerada pelo código do Listing 2 com a solução exata



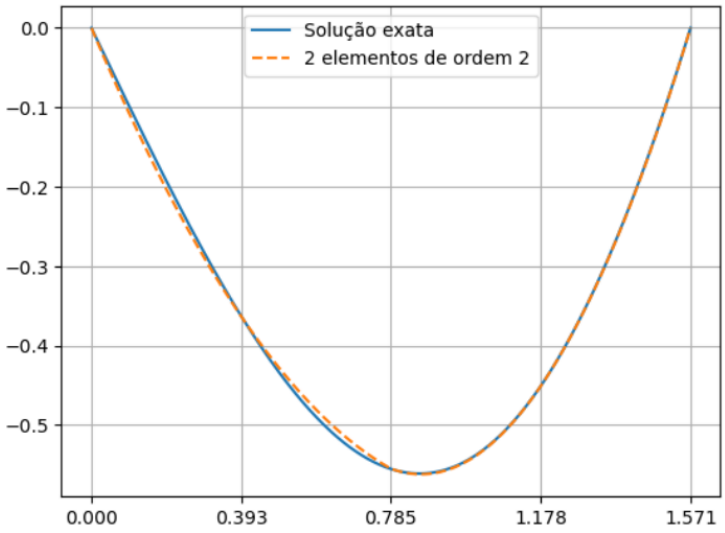
Fonte: Elaborada pelo autor

Figura 5 – Funções de forma de grau 2 geradas para o problema $u'' + u = 2\sin(x)$



Fonte: Elaborada pelo autor

Figura 6 – Comparação da solução apeoximada com a exata para o problema $u'' + u = 2\sin(x)$



Fonte: Elaborada pelo autor

4.2 Versão 1 da rede neural de aprendizado profundo hierárquica

A primeira versão do algoritmo principal do projeto foi feita de forma orientada a objeto. Havia uma classe principal chamada `HiDeNN_for_FEM`, responsável por, entre outras rotinas, chamar as redes hierárquicas e uni-las de forma paralela. Além disso, essa classe executa o algoritmo de cálculo da função de perda e treinamento, assim como exposto na Seção 3.4.

Quanto ao código de treinamento da rede, pode ser resumido pela função `train` exibida em parte no Listing 3.

Os métodos `optim.SGD`, `backward`, `step`, `zero_grad`, `MSELoss`, dentre outros utilizados no código, fazem parte da biblioteca `PyTorch`, módulo focado em redes neurais implementado em *Python*.

Como já explicado, essa versão passou por algumas complicações e foi deixada de lado em prol de uma versão matematicamente mais correta e com menor uso de bibliotecas, para um maior domínio do procedimento. Algumas tentativas falhas antes de ser percebido o erro matemático explicado na Seção 3.4 foram feitas. Dentre elas:

- Alterações de várias das funções utilizadas de bibliotecas diferentes para funções análogas da biblioteca `PyTorch`, em busca de uniformizar o código e evitar incompatibilidades.
- Diminuição do número de objetos da classe `torch.tensor` a fim de não ocorrer o problema dos seus gradientes se perderem na execução do *backpropagation* (como em casos de *vanishing gradient*).

Listing 3 – Parte da definição da função `train`

```
def train(self, epochs=1, lossfunc = nn.MSELoss(), lr=1e-3) : # utilizando um learning rate padrao de 0.001
    for epoch in range(epochs) :
        # -k * u''(x) + c * u'(x) + b * u(x) = f(x)
        u = self.forward().reshape(-1) # saida do foward da rede eh interpretada como u
        dudx = diff(u, self.dx) # derivada de u
        ddudx2 = diff(dudx, self.dx) # derivada segunda de u

        self.optimizer = torch.optim.SGD( [self . nodes] , lr=lr) # utilizando otimizador de descida do gradiente estocastico
        self.optimizer.zero_grad() # zerando gradiente
        loss = lossfunc (
            ( -self.k * ddudx2 + self.c * dudx [1:] + self.b * u[2:] ).requires_grad(True),
            tensor(self.f_func (self.dom [0,2:]))
            ) # calculo da funcao de perda
        loss.backward() # calculando backpropagation
        self.optimizer.step() # alterando os parametros da rede

        self.shape_func_arr = self.node_nets_func(torch.ones ((
        self.num_nodes, 1), dtype=torch.float64)) # gerando novas funcoes de forma com base nas novas posicoes dos nos

        self.displacement_arr = solve(self.K, self.F) # alpha = K^-1 x F
    # solucionando coeficientes interpoladores de cada funcao de forma

        self.u_aprox_arr = self.displacement_arr @ self.shape_func_arr # recalculando solucao aproximada do problema

        self.node_nets_arr = self.node_nets_func(self.displacement_arr) # gerando novas redes hierarquicas com base nos novos nos e novos coeficientes interpoladores
    return loss # retorna valor da perda final
```

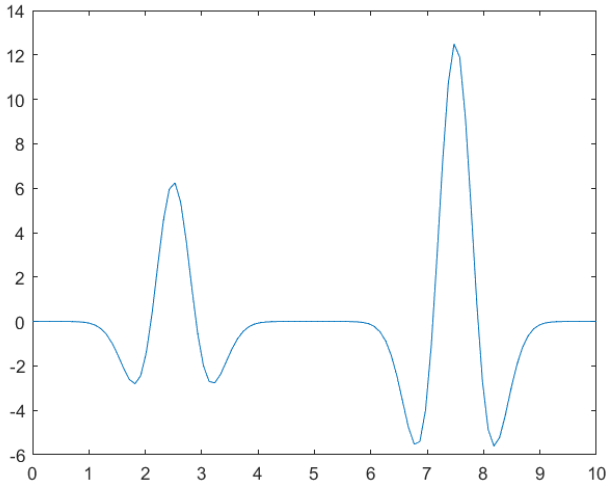
- Implementação das camadas da rede sem o uso das bibliotecas de redes neurais a fim de melhor entender o funcionamento do algoritmo.
- Reestruturação do código a fim de torná-lo mais simples e analisável.

Por mais que o treinamento da rede não tenha funcionado como o esperado, a inicialização do modelo para a formação da aproximação numérica utilizando funções chapéu funcionou. Com isso, a formulação do código ajudou a melhor consolidação das ideias e entendimento do problema.

4.3 Exemplo de aplicação da HiDeNN versão final

O problema de aplicação da HiDeNN é um dos exemplos do artigo Zhang et al. (2021a). Neste exemplo, há uma barra na posição horizontal bi-engastada nas suas duas

Figura 7 – Gráfico da função $b(x)$



Fonte: Elaborada pelo autor

extremidades e submetida a uma força vertical $b(x)$ (Equação 11).

$$b(x) = -\frac{4\pi^2(x-2.5)^2 - 2\pi}{e^{\pi(x-2.5)^2}} - \frac{8\pi^2(x-7.5)^2 - 4\pi}{e^{\pi(x-7.5)^2}} \tag{11}$$

O problema é descobrir o deslocamento vertical de cada ponto da barra ($u(x)$). Pela abordagem de elementos finitos, discretiza-se o domínio (Ω) em nós, descobre-se o deslocamento em cada um e utiliza-se esse deslocamento como interpolador da função de forma escolhida.

A barra possui um comprimento de $L = 10$, área transversal de $A = 1$ e módulo de Young de $E = 1$.

4.3.1 Implementação

Para a implementação da rede aplicada ao problema, inicia-se formulando a equação da energia potencial local do sistema (Equação 12), sendo Ω o domínio do sistema e $\mathbf{U}$ o conjunto dos deslocamentos. O primeiro termo da equação é originário da natureza elástica da barra, já o segundo o, o trabalho da força peso, contrária à elástica.

$$J(u) = \frac{1}{2}\int_\Omega AE\left(\frac{\mathrm{d}u}{\mathrm{d}x}\right)^2 \mathrm{d}x - \int_\Omega ub(x)\mathrm{d}x \quad \forall u \in \mathbf{U} \tag{12}$$

Assim, o objetivo é minimizar a equação da energia potencial local em todo o domínio, aproximando os deslocamentos por elementos finitos formados por redes neurais:

$$u^h(x) = \sum_{I=1}^{n} \mathcal{N}(\mathbf{W}, \mathbf{b}, \mathcal{A}, x)u_I \equiv \sum_I \mathcal{N}_I(x)u_I \tag{13}$$

Na Equação 13, $\mathbf{W}$ é o conjunto dos pesos da rede, $\mathbf{b}$, o conjunto dos viéses, e $\mathbf{A}$, as funções de ativação. Substituindo a rede interpoladora na equação da energia:

$$J(u_h) = \frac{1}{2}\int_\Omega \left(\frac{\mathrm{d}}{\mathrm{d}x}\left(\sum_I \mathcal{N}_I(x)u_I\right)\right)^2 \mathrm{d}x - \int_\Omega \left(\sum_I \mathcal{N}_I(x)u_I\right) b(x)\mathrm{d}x \tag{14}$$

Para minimizar a função de energia, é necessário que esta tenha sua derivada aproximada de 0, assim, $\frac{\partial J}{\partial u_I} = 0$. Assim, pode-se escrever:

$$\frac{\mathrm{d}}{\mathrm{d}u_I}\left[\frac{1}{2}\int_\Omega \left(\frac{\mathrm{d}}{\mathrm{d}x}\left(\sum_I \mathcal{N}_I(x)u_I\right)\right)^2 \mathrm{d}x\right] = \frac{\mathrm{d}}{\mathrm{d}u_I}\left[\int_\Omega \left(\sum_I \mathcal{N}_I(x)u_I\right) b(x)\mathrm{d}x\right] \tag{15}$$

Primeiramente, é analisado o termo do lado esquerdo. Considerando uma função de forma chapéu:

$$\begin{aligned}\frac{\partial}{\partial u_I}\left[\frac{1}{2}\int_\Omega \left(\frac{\mathrm{d}}{\mathrm{d}x}\left(\sum_I \mathcal{N}_I(x)u_I\right)\right)^2 \mathrm{d}x\right] &= \int_\Omega \left(\frac{\mathrm{d}\mathcal{N}_{I-1}(x)}{\mathrm{d}x}u_{I-1} + \frac{\mathrm{d}\mathcal{N}_I(x)}{\mathrm{d}x}u_I + \frac{\mathrm{d}\mathcal{N}_{I+1}(x)}{\mathrm{d}x}u_{I+1}\right) \\ &= -\frac{1}{\Delta x_I}u_{I-1} + \left(\frac{1}{\Delta x_I} + \frac{1}{\Delta x_{I+1}}\right)u_I - \frac{1}{\Delta x_{I+1}}u_{I+1}\end{aligned} \tag{16}$$

Sendo x_i a coordenada x do nó i, e $\Delta x_i = x_i - x_{i-1}$. Realizando esse procedimento com todos os nós do domínio, é possível escrever as equações na forma da multiplicação da matrix e vetor abaixo.

$$\begin{bmatrix} \frac{1}{\Delta x_1} + \frac{1}{\Delta x_2} & -\frac{1}{\Delta x_2} & 0 & \dots & 0 \\ -\frac{1}{\Delta x_2} & \ddots & \ddots & \ddots & \vdots \\ 0 & \ddots & \ddots & \ddots & 0 \\ \vdots & \ddots & \ddots & \ddots & -\frac{1}{\Delta x_{n-1}} \\ 0 & \dots & 0 & -\frac{1}{\Delta x_{n-1}} & \frac{1}{\Delta x_{n-1}} + \frac{1}{\Delta x_n} \end{bmatrix} \begin{Bmatrix} u_1 \\ \vdots \\ u_n \end{Bmatrix}$$

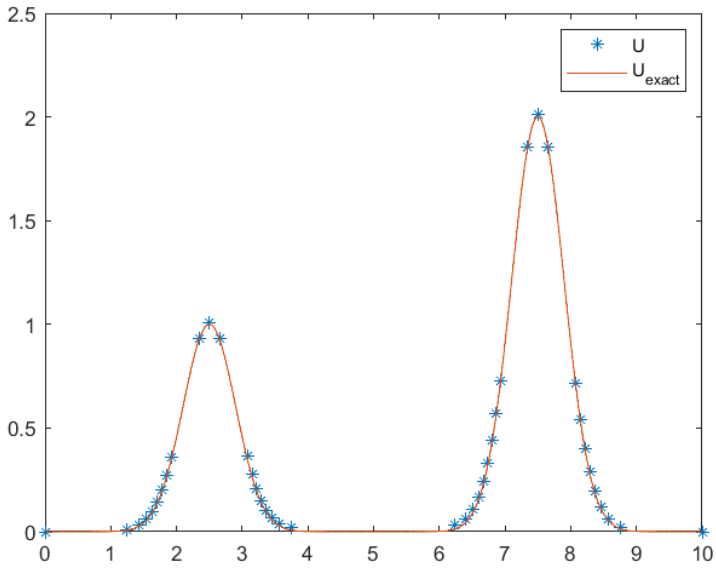
Percebe-se que essa matrix e vetor são análogos a $\mathbf{K}$ e $\mathbf{u}$, explicados na Seção 3.2. Analisando o termo da direita da Equação 15, percebe-se a analogia com o termo da Equação 7. Assim, é possível interpretar a minimização que a rede encontra da energia como a solução para a equação do método de elementos finitos.

4.3.2 Resultados

Conhecendo a solução exata, pode-se fazer algumas análises comparativas entre o resultado atingido e o esperado.

Percebe-se, pela Figura 8, que os nós de fato foram deslocados pelo treinamento, de maneira a se concentrar nos locais de maiores oscilações da solução, enquanto regiões

Figura 8 – Localização dos nós treinados pela rede sobre a solução exata



Fonte: Elaborada pelo autor

mais lineares apresentam menor densidade de nós.

$$||u - u^h||_{L^2} = \left(\frac{\int_\Omega (u - u^h)^2 \mathrm{d}x}{\int_\Omega (u^2) \mathrm{d}x} \right)^{1/2} \tag{17}$$

$$||u - u^h||_{H^1} = \left(\frac{\int_\Omega (u - u^h)^2 \mathrm{d}x + \int_\Omega \left(\frac{\mathrm{d}u}{\mathrm{d}x} - \frac{\mathrm{d}u^h}{\mathrm{d}x} \right)^2 \mathrm{d}x}{\int_\Omega u^2 \mathrm{d}x + \int_\Omega \left(\frac{\mathrm{d}u}{\mathrm{d}x} \right)^2} \right)^{1/2} \tag{18}$$

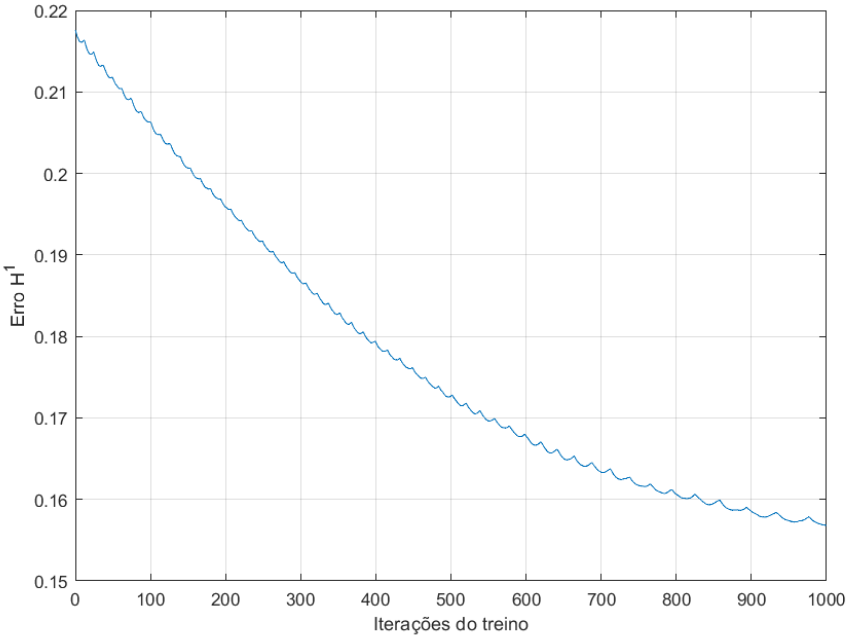
Para validar essa melhora no treinamento, é possível computar os erros dos tipos L^2 e H^1 (Equações 17 e 18) no decorrer do treinamento da rede e observar suas diminuições.

Para uma maior exploração dos parâmetros do otimizador da rede neural, foram alterados ρ_1 e ρ_2 de forma a analisar seus valores ótimos em relação a uma maior queda dos erros.

Primeiramente, foi alterado os valores desses parâmetros analisando o erro H^1 (Figuras 11 e 12). Nesses caso, ficou bem claro que de fato o $\rho_1 = 0.9$ recomendado foi vantajoso frente aos outros valores testados, pois foi mais eficiente em diminuir o erro. Já o $\rho_2 = 0.999$ teve um comportamento bem parecido com o $\rho_2 = 0.8$, sendo que ambos foram eficientes e não apresentaram vantagens tão nítidas entre si, mesmo quando analisados mais de perto (Figura 13).

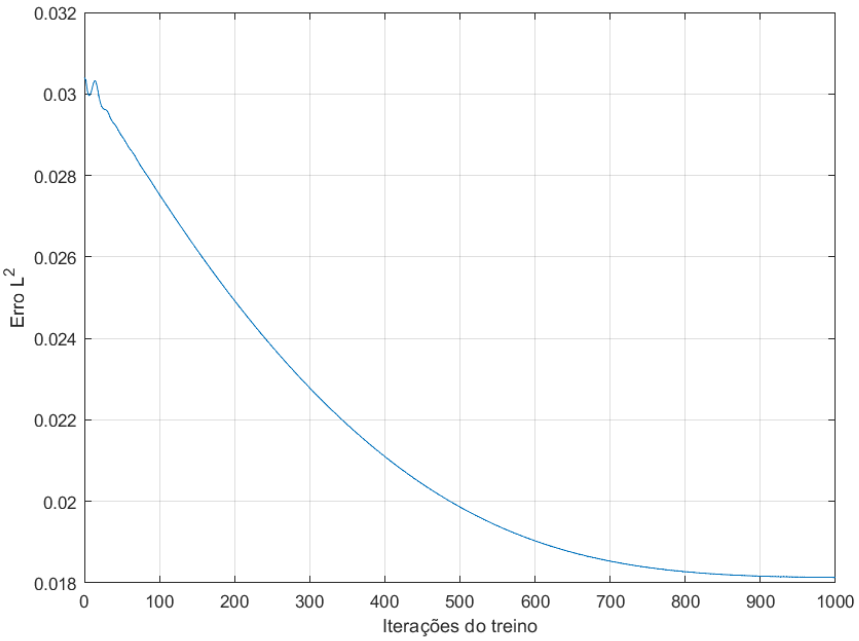
Realizando o mesmo procedimento analisando o erro L^2, observou-se uma consistência maior na descida do erro para $\rho_1 = 0.9$, assim como o recomendado, e novamente um empate entre $\rho_2 = 0.999$ e $\rho_2 = 0.8$

Figura 9 – Erro H^1 no decorrer do treinamento



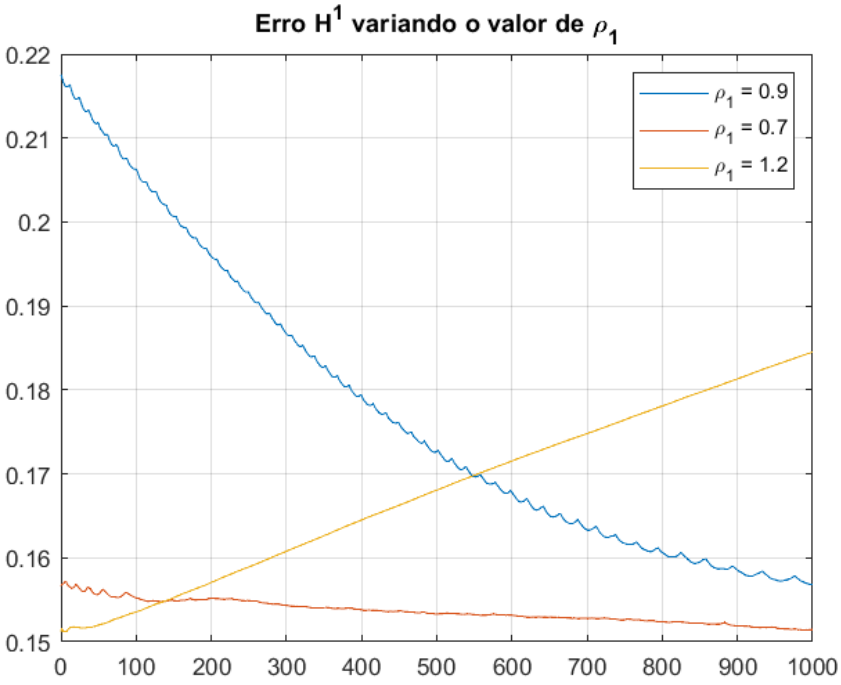
Fonte: Elaborada pelo autor

Figura 10 – Erro L^2 no decorrer do treinamento



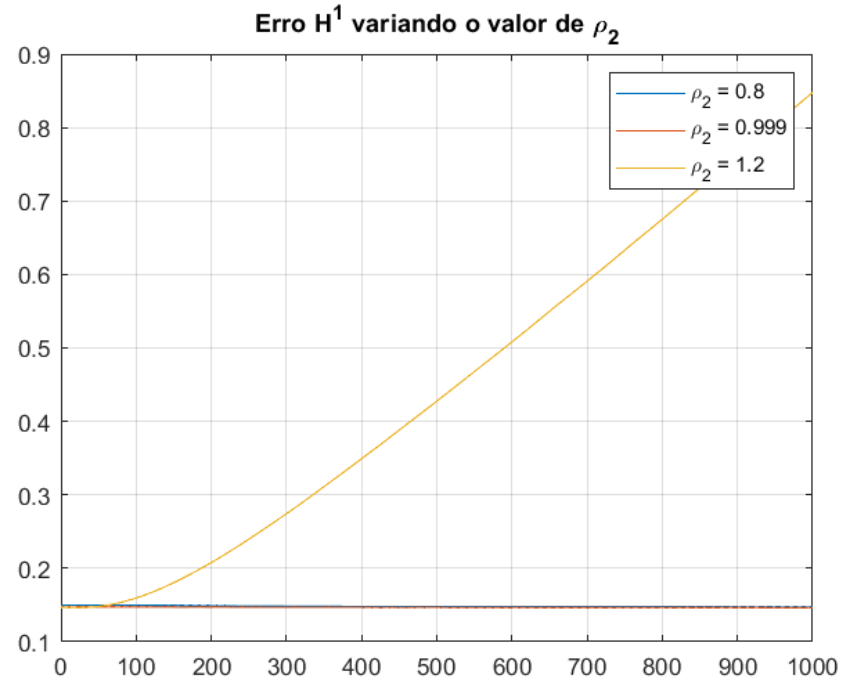
Fonte: Elaborada pelo autor

Figura 11 – Erro H^1 alterando ρ_1



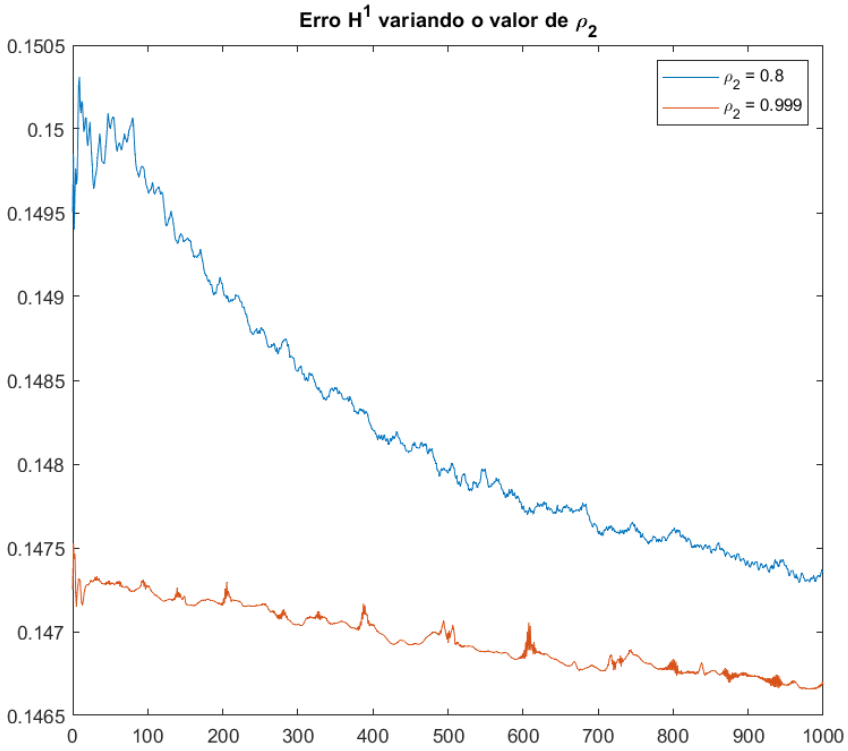
Fonte: Elaborada pelo autor

Figura 12 – Erro H^1 alterando ρ_2



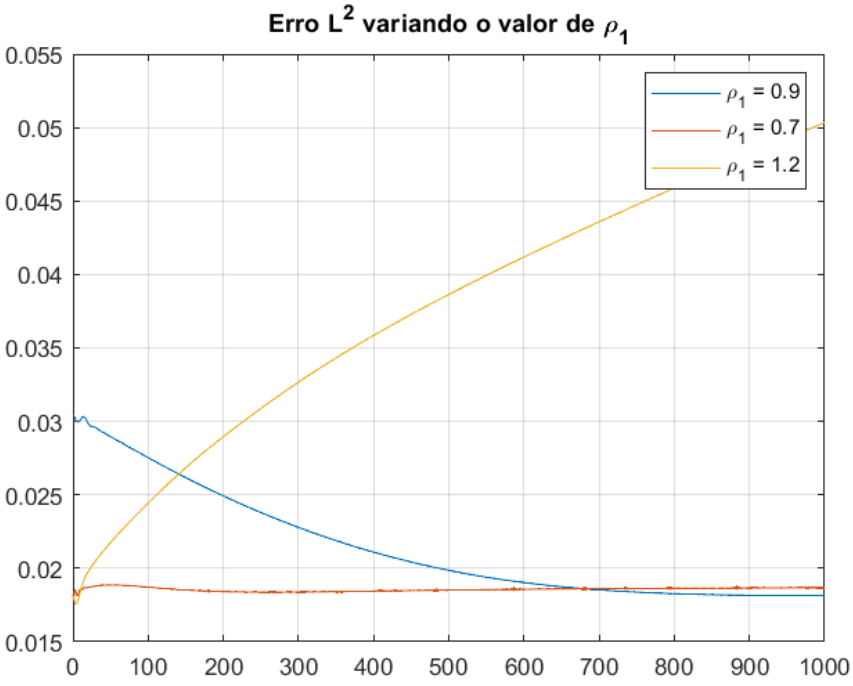
Fonte: Elaborada pelo autor

Figura 13 – Erro H^1 alterando ρ_2



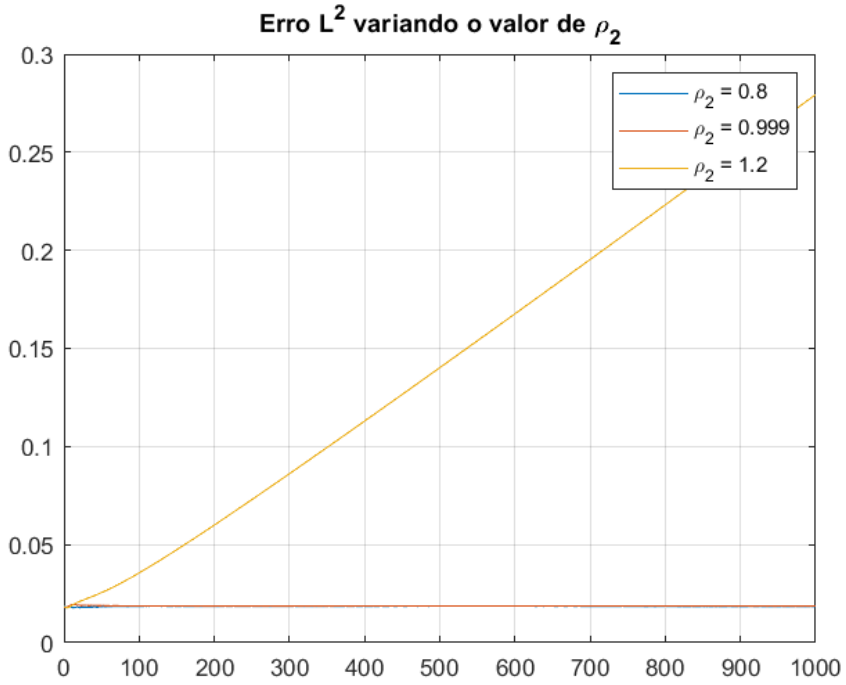
Fonte: Elaborada pelo autor

Figura 14 – Erro L^2 alterando ρ_1



Fonte: Elaborada pelo autor

Figura 15 – Erro L^2 alterando ρ_2



Fonte: Elaborada pelo autor

5 Conclusão

Como conclusão, tem-se que o modelo estudado consegue de fato encontrar uma solução próxima da solução exata e alterar as posições dos nós, assim como o desejado.

Nessa pesquisa, por meio de um exemplo simples, mas abrangente o suficiente para validar o funcionamento do modelo, foi possível analisar o seu erro considerando alterações nos parâmetros do otimizador `Adam`, otimizador este considerado estado da arte no campo de redes neurais. No exemplo exposto, que possui propositalmente pontos de derivada maior e outros com comportamento mais linear, foi possível observar que a rede preferencia uma concentração de nós nas regiões de maior derivada, o que corrobora com um menor erro, mesmo com menos nós.

Assim, é possível afirmar, pelo estudo, que o modelo analisado reflete a grande versatilidade da metodologia de redes neurais, que consegue funcionar bem mesmo em um contexto não relacionado a uma coleta de um grande bancos de dados, mas apenas utilizando equações matemáticas regidas pela física.

Referências

BARATTA, I. **FEniCSxCourse at ICMC23**. 2023. <https://github.com/IgorBaratta/FEniCSxCourse>. Accessed: 2024-7-20.

BECKER, E. B.; etc. **Finite Elements: v. 1: An Introduction**. Harlow, England: Longman Higher Education, 1981.

_____. _____. Harlow, England: Longman Higher Education, 1981. 40-59 p.

CROARKEN, M. Computing in britain during world war II. In: **IEE History of Technology Summer Meeting**. [S.l.: s.n.], 2002.

KINGMA, D. P.; BA, J. Adam: A method for stochastic optimization. 2014.

LANGTANGEN, H. P.; LINGE, S. **Finite difference computing with PDEs: A modern software approach**. Cham, Switzerland: Springer International Publishing, 2018.

LE-DUC, T.; NGUYEN-XUAN, H.; LEE, J. A finite-element-informed neural network for parametric simulation in structural mechanics. **Finite Elem. Anal. Des.**, v. 217, n. 103904, p. 103904, 2023.

MEETHAL, R. E.; KODAKKAL, A.; KHALIL, M.; GHANTASALA, A.; OBST, B.; BLETZINGER, K.-U.; WÜCHNER, R. Finite element method-enhanced neural network for forward and inverse problems. **Adv. Model. Simul. Eng. Sci.**, v. 10, n. 1, 2023.

NETZ, T.; DÜSTER, A.; HARTMANN, S. High-order finite elements compared to low-order mixed element formulations. **Z. Angew. Math. Mech.**, v. 93, n. 2-3, p. 163–176, 2013.

PANTIDIS, P.; MOBASHER, M. E. Integrated finite element neural network (I-FENN) for non-local continuum damage mechanics. **Comput. Methods Appl. Mech. Eng.**, v. 404, n. 115766, p. 115766, 2023.

RAO, S. S. THEORY OF VIBRATION | variational methods. In: **Encyclopedia of Vibration**. [S.l.]: Elsevier, 2001. p. 1344–1360.

SAHA, S.; GAN, Z.; CHENG, L.; GAO, J.; KAFKA, O. L.; XIE, X.; LI, H.; TAJDARI, M.; KIM, H. A.; LIU, W. K. Hierarchical deep learning neural network (HiDeNN): An artificial intelligence (AI) framework for computational science and engineering. **Comput. Methods Appl. Mech. Eng.**, v. 373, n. 113452, p. 113452, 2021.

SHARMA, P.; CHUNG, W. T.; AKOUSH, B.; IHME, M. A review of physics-informed machine learning in fluid mechanics. **Energies**, v. 16, n. 5, p. 2343, 2023.

SHI, Z.; GULGEC, N. S.; BERAHAS, A. S.; PAKZAD, S. N.; TAKAC, M. Finite difference neural networks: Fast prediction of partial differential equations. In: **2020 19th IEEE International Conference on Machine Learning and Applications (ICMLA)**. [S.l.]: IEEE, 2020. p. 130–135.

SOUSA, F. S.; LAGES, C. F.; ANSONI, J. L.; CASTELO, A.; SIMAO, A. A finite difference method with meshless interpolation for incompressible flows in non-graded tree-based grids. **J. Comput. Phys.**, v. 396, p. 848–866, 2019.

THOMÉE, V. The finite difference versus the finite element method for the solution of boundary value problems. **Bull. Aust. Math. Soc.**, v. 29, n. 2, p. 267–288, 1984.

TU, S. N. T.; NGUYEN, T. FinNet: Solving time-independent differential equations with finite difference neural network. 2022.

ZHANG, A.; LIPTON, Z. C.; LI, M.; SMOLA, A. J. Dive into deep learning. p. 170–186, 2021.

ZHANG, L.; CHENG, L.; LI, H.; GAO, J.; YU, C.; DOMEL, R.; YANG, Y.; TANG, S.; LIU, W. K. Hierarchical deep-learning neural networks: finite elements and beyond. **Comput. Mech.**, v. 67, n. 1, p. 207–230, 2021.